Group 15 – Assignment Two

Reliable Data Transfer Over UDP

CP372-C – Computer Networks – Fall 2026

Zakir Hasham | 169089154 | hash9154@mylaurier.ca
Ojas Chindarkar | 169103107| chin3107@mylaurier.ca
Danesh Jimmy Madan | 169107332| jimm7332@mylaurier.ca
Yushu Li | 169103977| lixx3977@mylaurier.ca

Introduction and System Overview

This assignment implements reliable file transfer over UDP using two application-layer protocols: **Stop-and-Wait** and **Go-Back-N (GBN)**, both of which we learned in lecture. To implement these protocols, we leveraged Python 3 and standard-library UDP sockets. Global to both protocols, a sender announces a file transfer, divides the file into DATA packets, and sends a completion packet after the file content. The receiver would then create the output file, write accepted DATA payloads in order, and close the file when the completion packet is acknowledged.

Since UDP sends independent datagrams without establishing a connection, it does not provide acknowledgments, retransmissions, or guaranteed delivery and ordering. As such, our file-transfer application needed additional capabilities to recover missing data and prevent duplicate or out-of-order packets from changing the received file. To accomplish this, we integrated sequence numbers to identify packets, acknowledgments to confirm reception, and sender timers to trigger retransmission when progress is not confirmed.

Looking at both protocol types, we noticed that Stop-and-Wait permits one unacknowledged packet at a time (in other words, the sender waits for the corresponding acknowledgment before advancing to the next packet). This allows for a simple transfer procedure but leaves the sender waiting between packets. GBN, in contrast, allows for multiple outstanding packets within a sliding window and uses cumulative acknowledgments to advance that window. If a timeout occurs, the sender retransmits all outstanding packets rather than only the missing packet.

Implemented Features

- File transfer using UDP sockets without third-party networking frameworks.
- Binary packet headers containing sequence number, acknowledgment number, and packet type.
- START and completion control packets for receiver file handling.
- Stop-and-Wait with one outstanding packet and timeout retransmission.
- GBN with a configurable sliding window, cumulative ACKs, and one retransmission timer.
- Receiver-side random packet loss, including loss of transfer control packets.
- Sender measurements of transfer time, throughput, and retransmitted packet count.

Stop-And-Wait Model Design

Program Structure

sender_stopwait.py reads the input file and sends each packet through send_and_wait(). This helper method only returns after receiving the expected ACK number. receiver_stopwait.py, the other endpoint of this protocol, receives datagrams in a continuous loop and retains the sender address, next expected DATA sequence number, last accepted sequence number, and completion state.

Both of our programs have been designed to default to UDP port 5001. Regarding IP addresses, the sender additionally defaults to destination 127.0.0.1, while the receiver binds to 127.0.0.1. The

submitted Stop-and-Wait receiver is therefore configured for testing on the same machine (which was a necessity for testing this assignment). Received files are saved under received/ using the supplied base filename.

Packet Format

The header uses struct format !III: three unsigned 32-bit integers in network byte order. The header is 12 bytes long. Payload bytes follow the header in the same UDP datagram.

Field	Size (in Bytes)	Meaning
Sequence Number	4	Identifier of the transmitted START, DATA, or COMPLETE packet
Acknowledgement Number	4	Sequence number being acknowledged; unused fields are set to 0
Packet Type	4	START = 0, DATA = 1, ACK = 2, COMPLETE = 3
Payload	Variable	UTF-8 filename for START, file bytes for DATA, or empty for ACK and COMPLETE

The START packet uses sequence number 0, while DATA packets begin at 1 and increment by 1 until all data is transmitted. Then, the COMPLETE packet follows this sequential order and uses the next number after the last DATA packet. Sequence numbers increase throughout a transfer rather than alternating between 0 and 1. Each acknowledgment, or ACK, packet uses sequence number 0 and carries the acknowledged sequence in its ACK-number field.

Additionally, the DATA payload is at most 1,024 bytes, so a full DATA datagram contains 1,036 application bytes, excluding the UDP and IP headers. The final DATA packet, however, may be shorter. No payload-length field is required because the payload occupies the remainder of the UDP datagram after the fixed header. No application-level checksum is included.

Timer Design and ACK Handling

Immediately after sending a packet with sock.sendto(), the sender then calculates a deadline using time.monotonic() and timeout. It sets the socket timeout to equal the remaining time and waits for a response. An acknowledgement is only accepted when it comes from the configured receiver address, has type ACK, and contains the expected acknowledgment number.

Please note that malformed or unrelated responses do not advance the transfer. Additionally, unexpected ACK numbers are ignored, and waiting continues within the original deadline. If no valid ACK arrives before the deadline, the sender retransmits the same packet and starts a new deadline. There is no fixed retry limit in the Stop-and-Wait sender.

The receiver accepts a DATA packet only when its sequence number equals expected_seq and a transfer is active. After writing its payload, the receiver advances expected_seq and acknowledges the accepted sequence number. Duplicate or out-of-order DATA is not written; the receiver repeats

the last accepted ACK when one is available. This prevents retransmitted DATA from creating duplicate content.

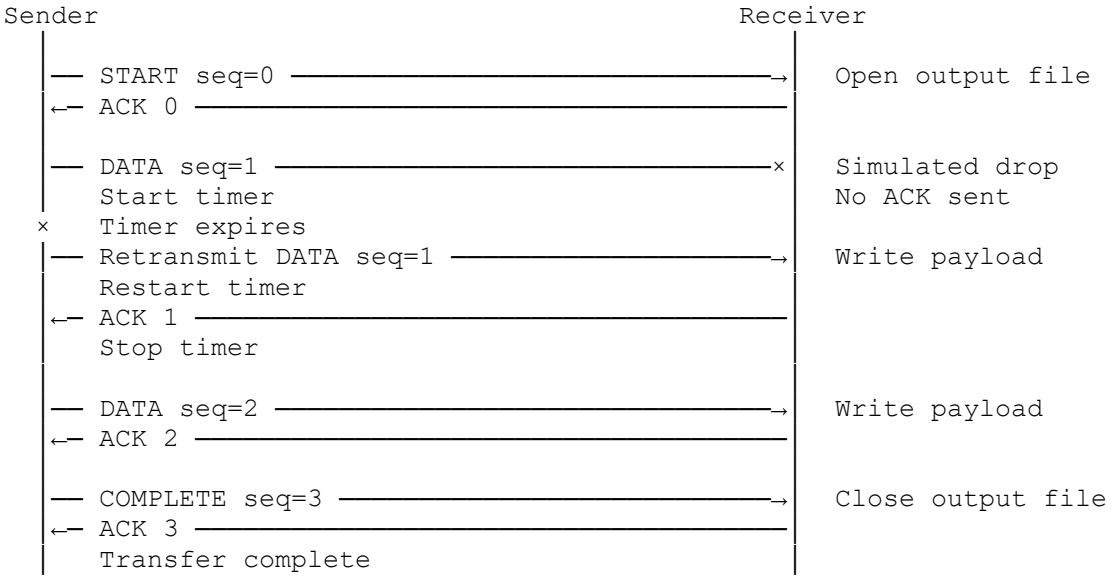
Window and File Transfer Workflow

The effective sender window is one packet. The next call to send_and_wait() does not begin until the current call returns after a matching ACK.

1. The sender transmits START with sequence 0 and the input filename.
2. Next, the receiver opens received/<filename> in binary write mode and sends ACK0.
3. The sender reads up to 1,024 bytes, packages them as DATA, and waits for that packet's ACK.
4. Receiver writes only the expected DATA payload and acknowledges it. Steps 3 and 4 repeat until the input ends.
5. Sender sends COMPLETE using the next sequence number and waits for its ACK.
6. Receiver accepts COMPLETE only at the expected sequence, closes the file, and acknowledges completion.

A repeated START during an active transfer from the same sender is acknowledged without reopening the file. A repeated COMPLETE after completion is acknowledged without reopening or rewriting the file. The receiver remains running after a transfer and can accept a new START.

Packet Loss Handling Flowchart



Go-Back-N Model Design

Program Structure

sender_gbn.py reads a supplied file or generates constant text content. It constructs START, DATA, and END packets in memory before entering the timed transmission loop. receiver_gbn.py tracks the next expected sequence number and writes only in-order packets.

The sender defaults to destination 127.0.0.1 and port 5001. The receiver defaults to bind address 0.0.0.0 and port 5001. Received files are stored as received/received_<filename>. GBN and Stop-and-Wait use different header layouts and type values, so each sender must be paired with its matching receiver.

Packet Format

The header uses struct format !BIIH: an unsigned 8-bit type, two unsigned 32-bit integers, and an unsigned 16-bit payload length. All multibyte fields use network byte order. The header is 11 bytes long.

Field	Size (in Bytes)	Meaning
Packet Type	1	START=1, DATA=2, ACK=3, END=4
Acknowledgement Number	4	Sequence number being acknowledged; unused fields are set to 0
Sequence Number	4	Identifier of the transmitted START, DATA, or END packet
Payload Length	2	Number of payload bytes following the header
Payload	Variable	UTF-8 filename for START, file bytes for DATA, or empty for ACK and END

START uses sequence 0, DATA begins at 1, and END follows the final DATA sequence. The default DATA payload is 1,024 bytes, making a full DATA datagram 1,035 application bytes before UDP and IP headers. ACK packets carry the acknowledged sequence number in the ACK-number field. No application-level checksum is included.

Timer Design and ACK Handling

The sender uses one timer, represented by timer_start, for the oldest unacknowledged packet. It starts the timer when sending the first packet into an empty outstanding window. The remaining wait is calculated using time.perf_counter() and the configured timeout.

An ACK advances the window only when base <= acknum < nextseqnum. On a valid cumulative ACK, the sender sets base = acknum + 1. ACK n therefore confirms reception of every packet through n. Duplicate or stale ACKs do not advance the window.

When an ACK leaves packets outstanding, the timer is restarted for the new base. When no packets remain outstanding, the timer is stopped. On timeout, the sender retransmits every packet in the range base through nextseqnum - 1 and restarts the timer. Retransmission counts increase by the number of packets resent, not by one per timeout. The sender exits with an error if its configured consecutive-timeout limit is exceeded without ACK progress.

Sliding Window Implementation

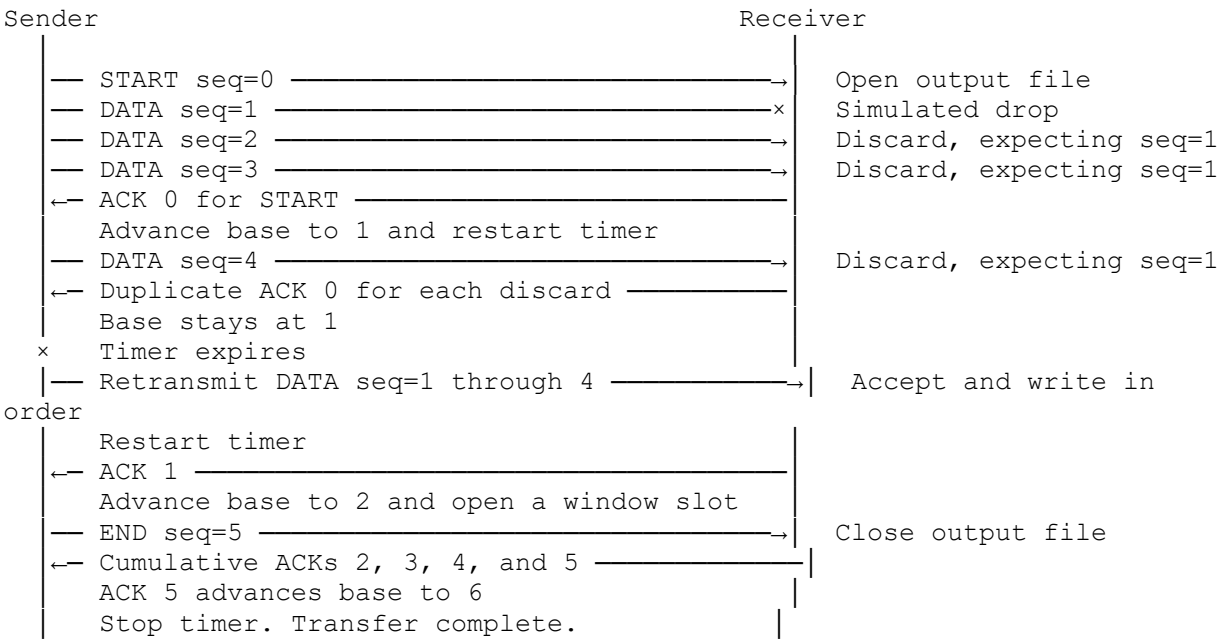
The sender maintains three variables:

Variable	Meaning
base	Oldest unacknowledged packet
nextseqnum	Next packet to send for the first time
window_size	Maximum number of outstanding packets

New packets may be sent while nextseqnum < base + window_size and unsent packets remain. The default window size is 4. START and END participate in the same sequence space and window as DATA.

For example, the sender can transmit packets 0, 1, 2, and 3 before waiting for an ACK. ACK0 advances base to 1 and makes room for packet 4. ACK1 and ACK2 similarly make room for packets 5 and 6. The sender therefore advances the window as acknowledgments arrive rather than waiting for the entire original window to finish.

Packet Loss Handling Flowchart



Experimental Results

We ran both protocols on Linux localhost using 1,024-byte DATA payloads and a 1 ms timeout. Go-Back-N used a window of four packets. Each table entry is the average transfer time in seconds over five trials.

(See Next Page)

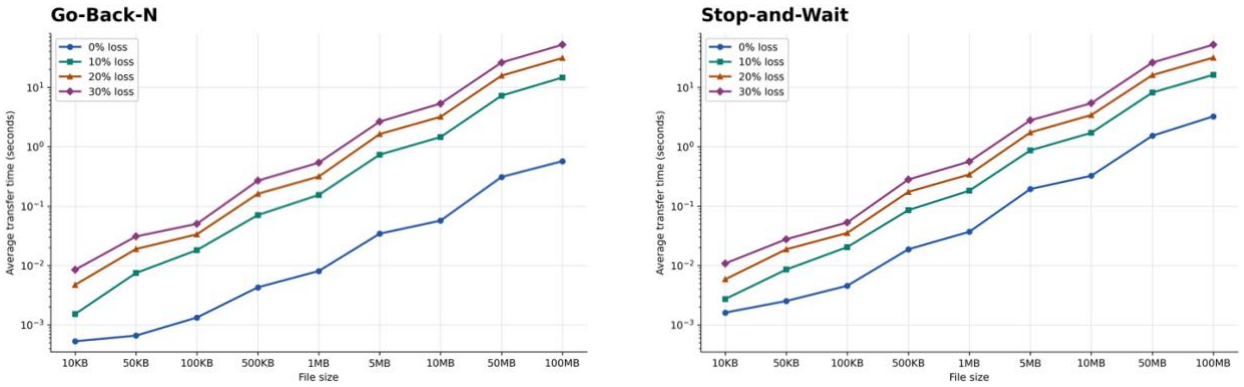
Average Stop-and-Wait Transfer Time (Average of Min 5 Tests Per File Size Per Loss Rate)

Loss	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.00161	0.00252	0.00455	0.01872	0.03687	0.19344	0.32339	1.52600	3.23481
10%	0.00272	0.00853	0.02039	0.08530	0.18138	0.86508	1.70945	8.14197	16.18866
20%	0.00584	0.01870	0.03522	0.17302	0.34023	1.73305	3.41202	16.00659	31.53460
30%	0.01082	0.02766	0.05291	0.27943	0.56158	2.77672	5.39213	26.04135	51.98599

Average GBN Transfer Time (Average of Min 5 Tests Per File Size Per Loss Rate)

Loss	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.00053	0.00066	0.00133	0.00429	0.00807	0.03432	0.0569172	0.30871	0.56889
10%	0.00152	0.00747	0.01816	0.07056	0.15397	0.73225	1.4493966	7.21536	14.56962
20%	0.00472	0.01895	0.03345	0.16081	0.31329	1.62555	3.1781934	15.65150	31.07836
30%	0.00848	0.03084	0.05024	0.26721	0.53669	2.63582	5.3213868	26.07629	51.96345

Graphics and Experimental Data Visualization



As we can observe from the data graphs, the transfer time increased with file size and packet loss for both protocols. At 0% loss, Go-Back-N transferred the 100 MB file in 0.57 seconds compared to 3.23 seconds for Stop-and-Wait. At 30% loss, both protocols averaged about 52 seconds for the 100 MB file. Because our tests ran on localhost, the round-trip time was close to zero, so the 1 ms retransmission timeout was the dominant cost of each lost packet. As a result, both protocols spent most of their time waiting for timeouts rather than transmitting. At high loss rates this timeout cost outweighed the advantage GBN gains from sending multiple packets before waiting for an ACK, so the two results converged.

Analysis & Discussion

Is Go Back N Always Faster?

No. Go-Back-N was faster in 33 of the 36 conditions we tested. Stop-and-Wait was faster for the 50 KB file at 20% and 30% loss and for the 50 MB file at 30% loss. GBN usually saves time by sending multiple packets while waiting for ACKs. When a packet is lost, however, the receiver discards later packets that arrive out of order, and the sender may have to resend several packets after a timeout. This reduces the benefit of pipelining.

Impact of Increasing Packet Loss?

Increasing packet loss made both protocols slower and reduced their throughput. For the 100 MB file, raising loss from 0% to 30% increased the average transfer time from 3.23 to 51.99 seconds for Stop-and-Wait and from 0.57 to 51.96 seconds for GBN. Larger files require more packets, so there are more opportunities for losses and timeout waits to accumulate.

Trade-Offs Between Simplicity and Efficiency?

Stop-and-Wait is simpler because it tracks only one outstanding packet and its ACK. The trade-off is that the sender sits idle while waiting for each ACK. GBN keeps the channel busier by allowing multiple packets in flight, but the sender must maintain a sliding window, process cumulative ACKs, and keep more packets available for retransmission. When a timeout occurs, Stop-and-Wait resends one packet, while GBN resends all outstanding packets in the window.

Situations Where Stop-and-Wait May Still Be Useful?

Stop-and-Wait is useful for small or infrequent transfers, especially when ACKs return quickly or the sender has limited memory. With only a few packets to send, the extra waiting can be small. The sender also only needs to keep one packet available for retransmission, which makes the protocol easier to implement and debug.

Conclusion

In this assignment, we used sequence numbers, ACKs, and retransmissions to transfer files reliably over UDP under simulated packet loss. Go-Back-N was generally faster because it could send multiple packets before waiting for an ACK. Its advantage became smaller as packet loss increased. At 30% loss, both protocols took about 52 seconds to transfer the 100 MB file. At the end of the day, we conclude that Stop-and-Wait remains a practical choice for small transfers where keeping the implementation simple matters more than speed. Moving forward, to improve throughput, we could tune the timeout based on how long ACKs take to arrive, so the sender can recover from packet loss sooner without resending packets unnecessarily.